richbeam

LakiBeam SDK User Manual

Contents

1 Architecture 4
2 Instructions for Use 5
2.1 Environment requirements 5
2.2 Development Environment Building (windows) 5
3 Class Specification (UDP protocol) 9
3.1 Structure Definition 9
3.2 Function descrIPtion 10
3.2.1 Constructor Function 10
3.2.2 Destructor Function 10
3.2.3 dots_valid() 10
3.2.4 get_pack() 11
3.2.5 restart() 11
3.2.6 wait() 11
4 Class Specification(HTTP protocol) 12
4.1 Class definition 12
4.2 Function descrIPtion 12
4.2.1 delete_override() 12
4.2.2 get_filter_level() 13
4.2.3 get_firemware() 13
4.2.4 get_host() 13
4.2.5 get_host_IP() 14
4.2.6 get_host_port() 14
4.2.7 get_laser_enable() 14
4.2.8 get_laser_start() 15
4.2.9 get_laser_stop() 15
4.2.10 get_monitor() 15
4.2.11 get_motor_rpm() 16
4.2.12 get_network() 16
4.2.13 get_overview() 17
4.2.14 get_resolution() 17
4.2.15 get_scan_range() 18
4.2.16 get_scanfreq() 18
4.2.17 put_filter_level() 18
4.2.18 put_host_IP() 19
4.2.19 put_host_port() 19
4.2.20 put_laser_enable() 19
4.2.21 put_laser_start() 20
4.2.22 put_laser_stop() 20
4.2.23 put_override() 21
4.2.24 put_reset() 21
4.2.25 put_scanfreq() 22
5 The SDK development process 23

5.1 lidar active upload data mode process 23

1 Architecture

The data acquisition system of LakiBeam includes the following parts from the software perspective:

(1) LakiBeam: Collect environmental data and information.

(2) Ethernet: Lidar is connected to PC via the Ethernet. Collected data by lidar is sent to the PC through the UDP protocol.

(3) SDK: It can receive the data collected by the LakiBeam through the interface provided by the SDK and configure the lidar parameters, IP address, etc.

2 Instructions for Use

2.1 Environment requirements

This SDK is the source code, please integrate into the target system.

SDK uses the Boost library, please prepare the boost library before compiling, Boost library version recommended: 1.63 and above.

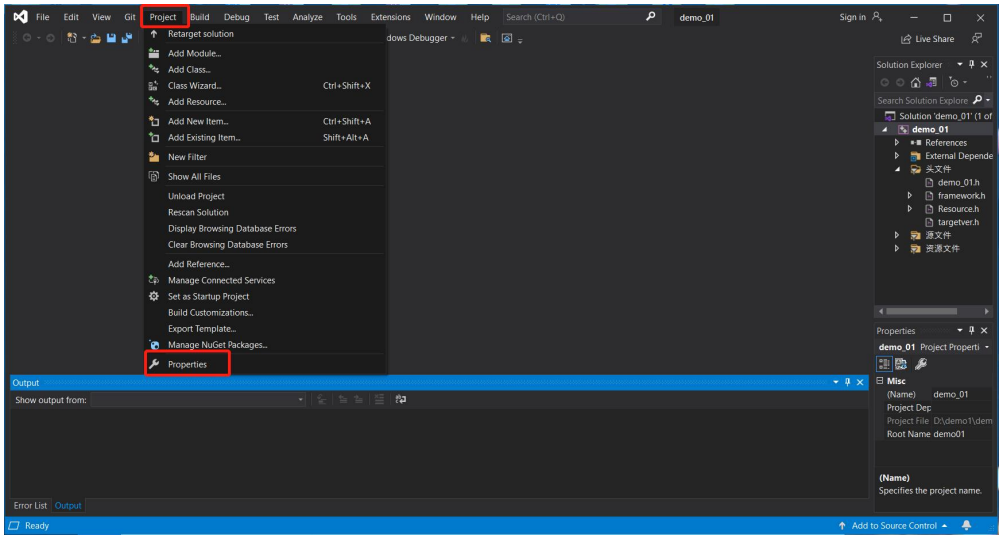
2.2 Development Environment Building

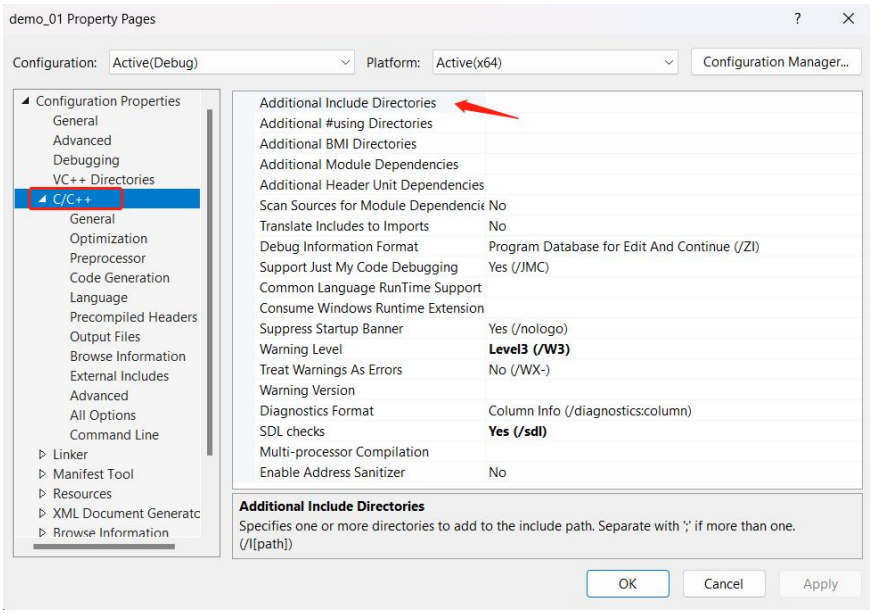
Windows (Visual Studio):

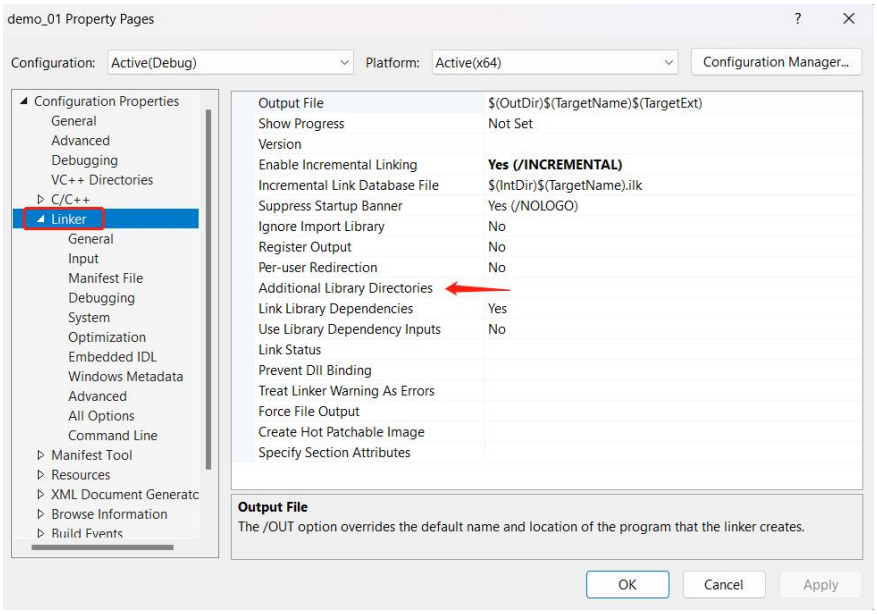
Based on the Windows operating system, LakiBeam SDK development environment should follow conditions below: recommended: Visual Studio 2012 or above installed and compiled boost library installed.

(1) Configure compilation options;

a. Open the project properties panel:







b. In the left list, select C / C + + and Linker, respectively

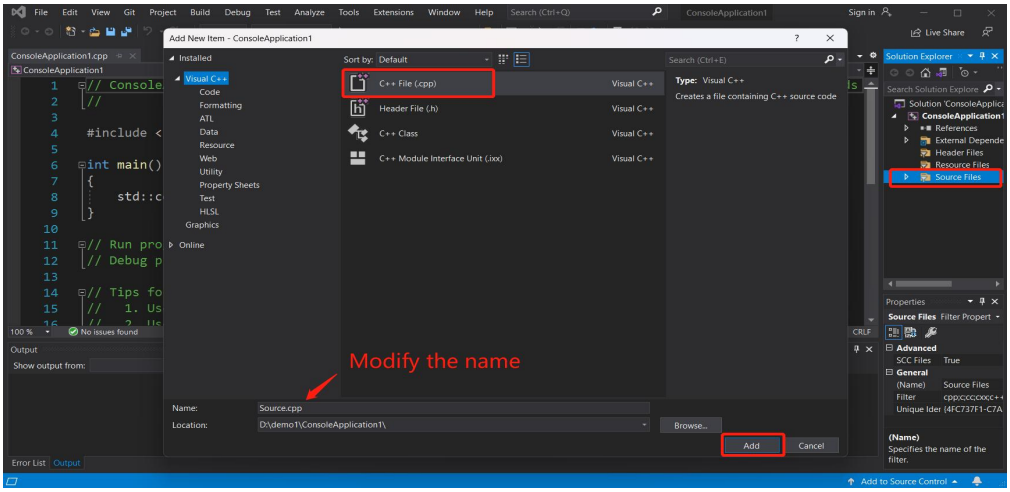
c. Select the additional inclusion directory and additional library directory on the right and click the drop-down button

d. Select the Edit page

e. Add the boost dependency to the list

f. Click "Apply" to complete the configuration

(2) Create a source file



(3) Add code : Put the code in the LakiBeamUDP.cpp file in the editor that pops up

(4) Compile running : In the main menu of Visual Studio, click Debug-> Start Debug to compile and run your program

Ubuntu (Visual Studio Code):

(1) Install and compile the boost library

a. Download boost(version 1.63),take version 1.78 as an example

b. UnzIP the package

c. Open terminal

cd boost_1_78_0 // enter into the file,boost_1_78_0 is the folder name //

./bootstrap.sh // executive command //

./b2 install --prefix=/home/fen/boost // home/fen/boost is your own installation path, you can create a new folder as the installation path //

cd ~/boost // enter into the new boost directory //

ls // there will be an include folder and a lib folder,one for header files and one for library files //

sudo mv -f ./lib/* /usr/lib //move the library file to /usr/lib,it doesn't matter if it can't be moved, continue to the next step //

sudo cp -rf ./include/boost /usr/include // copy the header file to /usr/include,boost compilation is complete //

(2) Obtain SDK

a. Decompress the SDK (the SDK archive is .rar, if it cannot be decompressed, follow the steps below, if you can decompress, skIP directly to the third step)

b. Open Terminal

sudo apt-get install rar

sudo apt-get install unrar

(3) Test Code

a. Download the VScode compilation tool

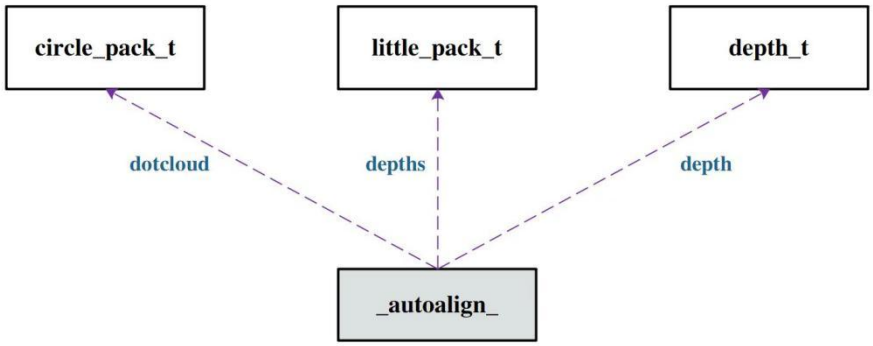
b. Open the SDK in VScode

c. Run and debug

3 Class Specification (UDP protocol)

3.1 Structure Definition

Collaborative diagram of the _autoalign_ structure:



circle_pack_t structure: A repackaged depth packet minimum cell
depth_t structure: The minimum parcel of the lidar depth data
little_pack_t structure: The minimum unit of the data structure for a frame of lidar depth data
repark_t structure: A frame represents a piece of depth data
udp_pack_t structure: The data structure of one-frame UDP packet
LakiBeamUDP: Lidar UDP packets are unpacked into packets, receive the depth data of lidar and reassemble the data blocks, each data block contains a circle of depth data, and the blocks are connected sequentially between blocks.

3.2 Function description

3.2.1 Constructor Function

LakiBeamUDP( )

LakiBeamUDP::LakiBeamUDP (

string local IP,

string local port,

string laser IP,

string laser port)

parameter:

local IP: local IP

local port:: local port number

laser IP: lidar IP

laser port:: lidar port number

3.2.2 Destructor Function

~ LakiBeamUDP( )

LakiBeamUDP::~ LakiBeamUDP ( )

3.2.3 dots_valid()

u32_t LakiBeamUDP::dots_valid (void)

Get the valid value of the lidar package

returned value:

Effective number of lidar data

3.2.4 get_pack()

bool LakiBeamUDP::get_pack (repark_t & pack)

Get the lidar data packet

parameter:

pack: lidar data return value, including time stamp, angle value, depth data and echo strength

returned value:

true: operation succeeded

false: operation failed

3.2.5 restart()

void LakiBeamUDP::restart (void)

Restart the SDK function package of the lidar UDP

3.2.6 wait()

void LakiBeamUDP::wait (void)

Wait for a new package of lidar data

4 Class Specification(HTTP protocol)

4.1 Class definition

Class filter_c: lidar filter level class, filter parameters
Class firmware_c: obtaining the lidar firmware information package class
Class host_c: host configuration class, configuration parameters of the host
Class monitor_c: obtain system monitoring data packaging and system data information
Class network_c: network information class encapsulation, network information parameters
Class overview_c: reset the server data class, the server parameters
Class scan_range_c: acquire the lidar angle scan range package class, the lidar scan start position and end position
Class LakiBeamHTTP: lidar profile class, which can configure all the lidar operating parameters

4.2 Function description

4.2.1 delete_override()

bool LakiBeamHTTP::delete_override (string & result)

Remove the static mode configuration and set to DHCP mode

parameter:

result: Result of this operation

returned value:

true: Operation succeeded

false: Operation failed

4.2.2 get_filter_level()

bool LakiBeamHTTP::get_filter_level (filter_c & level)

Set the lidar filter level

parameter:

level: filter information structure return value

returned value:

true: Operation succeeded

false: Operation failed

4.2.3 get_firmware()

bool LakiBeamHTTP::get_firmware (firmware c & firmware)

Get the lidar firmware information

parameter:

firmware: Reference class parameters for lidar firmware information

returned value:

true: Operation succeeded

false: Operation failed

4.2.4 get_host()

bool LakiBeamHTTP::get_host (host c & host)

Gets the current lidar Host terminal IP configuration

parameter:

host : host IP configuration information return value

returned value:

true: Operation succeeded

false: Operation failed

4.2.5 get_host_IP()

bool LakiBeamHTTP::get_host_IP (string & IP)

Get the current lidar Host terminal IP address

parameter:

IP: The IP address return value of the lidar Host terminal

returned value:

true: Operation succeeded

false: Operation failed

4.2.6 get_host_port()

bool LakiBeamHTTP::get_host_port (string & port)

Get the IP address port number of the current lidar Host terminal

parameter:

port : Host IP address port number return value

returned value:

true: Operation succeeded

false: Operation failed

4.2.7 get_laser_enable()

bool LakiBeamHTTP::get_laser_enable (string & laser state)

Obtain the laser state:

parameter:

motor rpm: Current laser state return value (true: enable false: disabled)

returned value:

true: Operation succeeded

false: Operation failed

4.2.8 get_laser_start()

bool LakiBeamHTTP::get_laser_start (string & start)

Obtain the laser scanning start angle

parameter:

start: Lidar scanning starting angle return value (unit: degree)

returned value:

true: Operation succeeded

false: Operation failed

4.2.9 get_laser_stop()

bool LakiBeamHTTP::get_laser_stop (string & stop)

Obtain the laser scanning end angle

parameter:

start: Lidar scan end angle return value (unit: degree)

returned value:

true: Operation succeeded

false: Operation failed

4.2.10 get_monitor()

bool LakiBeamHTTP::get_monitor (monitor c & monitor)

Obtain the lidar monitoring information

parameter:

monitor: Obtain the system monitoring data, including the average system payload, memory usage, and system running time

returned value:

true: Operation succeeded

false: Operation failed

4.2.11 get_motor_rpm()

bool LakiBeamHTTP::get_motor_rpm (string & motor rpm)

Obtain the lidar real-time rotational speed

parameter:

motor _ rpm: the current lidar speed return value (unit: rpm)

returned value:

true: Operation succeeded

false: Operation failed

4.2.12 get_network()

bool LakiBeamHTTP::get_network (network c & network)

Get network information

parameter:

network: Network information return value, including Ethernet status, duplex mode, MAC address, host name, IPv4 information, and Ethernet transmission rate

returned value:

true: Operation succeeded

false: Operation failed

4.2.13 get_overview()

bool LakiBeamHTTP::get_motor_rpm (string & motor rpm)

Obtain the lidar real-time rotational speed

parameter:

overview: Including lidar scanning frequency, lidar real-time speed, laser switch status, resolution, scanning Angle range, filter level, Host terminal IP address and port number

returned value:

true: Operation succeeded

false: Operation failed

4.2.14 get_resolution()

bool LakiBeamHTTP::get_resolution (string & resolution)

Obtain the current horizontal angular resolution of the lidar

parameter:

resolution: lidar horizontal angular resolution return value, the corresponding horizontal angular resolution of each scanning frequency is: 10Hz / 0.1°, 20Hz / 0.25°, 25Hz / 0.25°, 30Hz / 0.25°

returned value:

true: Operation succeeded

false: Operation failed

4.2.15 get_scan_range()

bool LakiBeamHTTP::get_scan_range (scan range c & scan range)

Obtain the lidar scan range

parameter:

scan_range: lidar scan range return value (unit: degrees)

returned value:

true: Operation succeeded

false: Operation failed

4.2.16 get_scanfreq()

bool LakiBeamHTTP::get_scanfreq (string & result)

Get the lidar scan frequency

parameter:

result: Current lidar scanning frequency return value (unit: Hz)

returned value:

true: Operation succeeded

false: Operation failed

4.2.17 put_filter_level()

bool LakiBeamHTTP::put_filter_level (string & level)

Set the current lidar data filter unit level

parameter:

level: Set the filter level, divided into 0~3 in a total of 4 levels, 0 is closed

returned value:

true: Operation succeeded

false: Operation failed

4.2.18 put_host_IP()

bool LakiBeamHTTP::put_host_IP (string & IP)

Set the current lidar Host terminal IP address

parameter:

IP: the IP address setting value of the lidar Host terminal

returned value:

true: Operation succeeded

false: Operation failed

4.2.19 put_host_port()

bool LakiBeamHTTP::put_host_port (string & IP)

Set the IP address port number of the current lidar Host terminal

parameter:

IP: Host terminal IP address port number setting value

returned value:

true: Operation succeeded

false: Operation failed

4.2.20 put_laser_enable()

bool LakiBeamHTTP::put_laser_enable (string & config state)

Switch laser status

parameter:

config_state: Laser state set point (true: enable false: disabled)

returned value:

true: Operation succeeded

false: Operation failed

4.2.21 put_laser_start()

bool LakiBeamHTTP::put_laser_start (string & start)

Set the laser scan start angle

parameter:

start: Set the setting range: 0 degrees 315 degrees Note: Set the start angle should be less than the end angle, otherwise the setting fails

returned value:

true: Operation succeeded

false: Operation failed

4.2.22 put_laser_stop()

bool LakiBeamHTTP::put_laser_stop (string & stop)

Set the laser scan end angle

parameter:

stop: Set the lidar scan end Angle setting range: 0 degrees 315 degrees (**Note**: the set starting Angle should be less than the end Angle, otherwise the setting fails)

returned value:

true: Operation succeeded

false: Operation failed

4.2.23 put_override()

bool LakiBeamHTTP::put_override (string override, string & result)

Gets the current static IP rewrite value

parameter:

override: Static IP override address settings, valid when DHCP mode is open, otherwise it is invalid

result: The result or error information of this operation

returned value:

true: Operation succeeded

false: Operation failed

4.2.24 put_reset()

bool LakiBeamHTTP::put_reset (string & result)

Set system reset

parameter:

result: Result of this operation

returned value:

true: Operation succeeded

false: Operation failed

4.2.25 put_scanfreq()

bool LakiBeamHTTP::put_scanfreq (string & freq)

Set the lidar scanning frequency

parameter:

freq: Set scan frequency (option: 10 20 25 30, Recommended value: 25)

returned value:

true: Operation succeeded

false: Operation failed

5 The SDK development process

5.1 lidar active upload data mode process

The point cloud data of LakiBeam lidar is acquired by the lidar.

lidar active upload data mode refers to setting the network port IP connected to the lidar according to the configuration of Host IP and Data Port in the lidar, opening the port for receiving the data according to the Host IP and port number in the configuration interface, and connecting the lidar according to the lidar IP, so as to obtain the lidar active upload data.

main points:

(1) Know the Host IP and Data Port information in the lidar configuration interface;

(2) Set the network IP connected to the lidar as Host IP in the configuration interface;

(3) Set the port for receiving the data according to the Host IP and Data Port in the lidar configuration interface.

At the beginning of lidar charging, the default data transmission mode is active and data upload mode. At this time, the upper computer can use the lidar active data upload mode to obtain data. In the process of lidar active upload data mode, the lidar can be connected through the USB Type-C cable, and the browser can log in to the lidar configuration interface to view the Host IP and Data Port configuration information in the lidar, as shown in Figure 5.1.

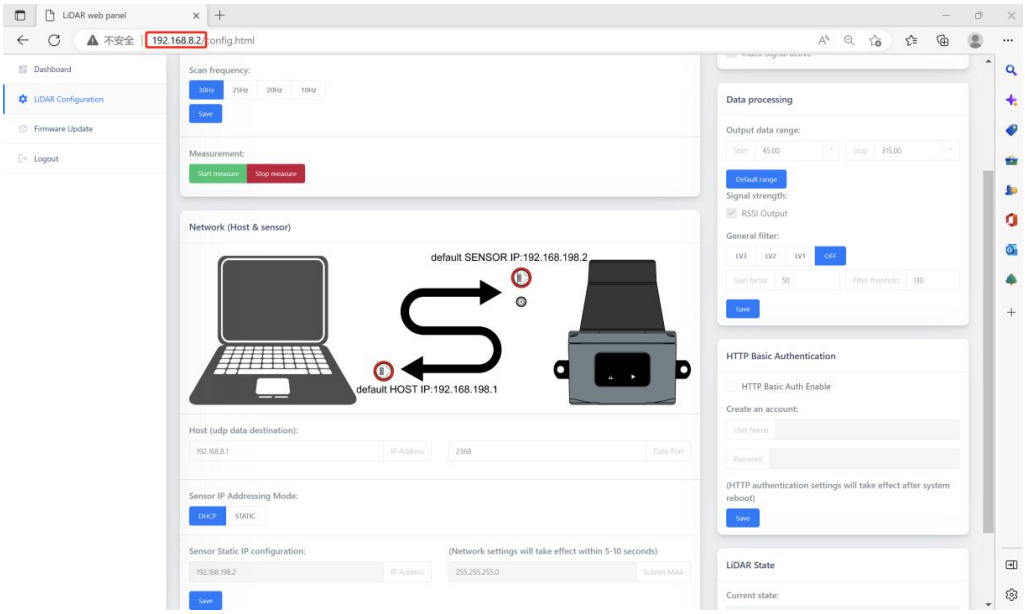


Figure 5.1 Lidar Configuration Interface

When the lidar configuration information needs to be modified, open the configuration interface as shown in Figure 5.1 to modify the lidar network configuration, requiring the Host IP and the lidar IP to be in the same network segment.

The initial lidar default settings are as follows:

Using process:

(1) Establish the equipment object, change the host IP and Data Port to acquire the lidar data;

(2) According to the lidar IP, connect to the lidar, and request the data transmission;

(3) Set the lidar parameters;

(4) Query the round for obtaining the data.